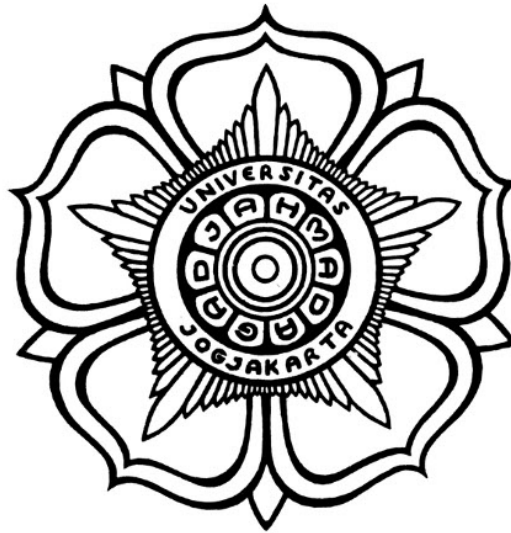


«JUDUL SKRIPSI»

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

«AUTHOR»

«NIM»

PROGRAM SARJANA PROGRAM STUDI «NAMA PRODI»
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
«TAHUN PENDADARAN»

HALAMAN PENGESAHAN

«JUDUL SKRIPSI»

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

«AUTHOR»

«NIM»

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Dosen Pembimbing 1, S.T., M.Eng., PhD.

«NIP xxxxxx»

Dosen Pembimbing 2, S.T., M.Eng., PhD.

«NIP xxxxxx»

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama :
NIM :
Tahun terdaftar :
Program : Sarjana
Program Studi :
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Nama Mahasiswa
NIM

HALAMAN PERSEMBAHAN

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

KATA PENGANTAR

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Orang 1 yang telah
2. Orang 2 yang telah
3. <isi dengan nama orang lainnya>

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi kita semua, aamiin.

Catatan: setiap nama yang dituliskan boleh disertai dengan alasan berterima kasih.

DAFTAR ISI

DAFTAR TABEL

DAFTAR GAMBAR

DAFTAR SINGKATAN

[SAMPLE]

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
$\mathbf{w}$	=	vektor bobot
$\mathbf{x}$	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Penelitian ini berfokus pada perancangan dan implementasi peripheral transmitter I2S (Inter-IC Sound) menggunakan bahasa Verilog HDL yang diintegrasikan ke dalam System-on-Chip (SoC) berbasis prosesor MicroBlaze V pada FPGA. Peripheral ini diimplementasikan sebagai custom IP dengan antarmuka AXI4-Lite slave sehingga dapat dikonfigurasi dan diisi data audio melalui register memory-mapped oleh perangkat lunak yang berjalan di Xilinx Vitis. Platform target yang digunakan adalah board FPGA Basys3, sedangkan keluaran audio direalisasikan melalui modul DAC PCM5102A. Modul pendukung seperti UART dan infrastruktur clock hanya digunakan sebagai penunjang sistem, sementara kontribusi utama skripsi ini adalah desain RTL, integrasi, dan validasi peripheral transmitter I2S.

Metodologi penelitian mencakup: (1) desain RTL Verilog untuk transmitter I2S yang mengikuti format Philips I2S untuk transmisi audio stereo, (2) perancangan antarmuka register AXI4-Lite untuk kontrol, pemuatan sampel, dan observasi status, (3) implementasi buffer berbasis FIFO dan interupsi watermark untuk mengurangi risiko underrun, (4) integrasi custom IP I2S ke dalam SoC MicroBlaze V menggunakan Vivado block design, (5) pengembangan perangkat lunak uji dalam bahasa C pada Xilinx Vitis untuk konfigurasi register, prefill FIFO, dan pengiriman sampel audio, serta (6) verifikasi melalui synthesis, implementation, pengamatan Integrated Logic Analyzer (ILA), dan pengujian hardware dengan modul PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S yang dirancang dapat diintegrasikan dengan baik ke dalam SoC berbasis MicroBlaze V dan dioperasikan melalui kontrol memory-mapped AXI4-Lite. Desain yang diimplementasikan mampu membangkitkan sinyal I2S yang diperlukan, mentransmisikan sampel audio stereo dalam format Philips I2S, serta mendukung dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz yang diturunkan dari konfigurasi audio clock. Validasi perangkat keras menunjukkan bahwa desain bekerja sesuai tujuan pada platform yang diuji dan dapat menjadi dasar yang dapat digunakan kembali untuk sistem keluaran audio digital berbasis FPGA.

Kata kunci : MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Audio Digital

ABSTRACT

This research focuses on the design and implementation of an I2S (Inter-IC Sound) transmitter peripheral using Verilog HDL, integrated into a System-on-Chip (SoC) based on the MicroBlaze V processor on FPGA. The peripheral is implemented as a custom AXI4-Lite slave IP so that it can be configured and supplied with audio data through memory-mapped registers from software running in Xilinx Vitis. The target platform is the Basys3 FPGA board, while audio output is generated through a PCM5102A DAC module. Supporting modules such as UART and clocking infrastructure are included only as system support, whereas the main contribution of the thesis is the RTL design, integration, and validation of the I2S transmitter peripheral.

The methodology includes: (1) Verilog RTL design of an I2S transmitter that follows the Philips I2S format for stereo audio transmission, (2) design of an AXI4-Lite register interface for control, sample loading, and status observation, (3) implementation of FIFO-based buffering and watermark interrupt support to reduce underrun risk, (4) integration of the custom I2S IP into a MicroBlaze V SoC using Vivado block design, (5) development of C test software in Xilinx Vitis for register configuration, FIFO prefill, and audio sample transmission, and (6) verification through synthesis, implementation, Integrated Logic Analyzer (ILA) observation, and hardware testing with the PCM5102A module.

The results show that the designed I2S transmitter IP can be integrated successfully into a MicroBlaze V-based SoC and operated through AXI4-Lite memory-mapped control. The implemented design is able to generate the required I2S signals, transmit stereo audio samples in Philips I2S format, and support two nominal sampling-rate modes around 48 kHz and 96 kHz derived from the audio clock configuration. Hardware validation indicates that the design functions as intended on the tested platform and provides a reusable basis for FPGA-based digital audio output systems.

Keywords: MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Digital Audio

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain philips_{i2s}1986. *Protokol ini menggunakan tiga sinyal utama, yaitu bit clock (BCLK)*

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi trimberger_{history_fpga}2018. *Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA menawarkan software co-design : fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara*

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V waterman_{riscv}2016 pada ekosistem Lite arm_{amba}xi2010, *peripheral dapat diakses dengan model memory-mapped I/O yang sederhana*

Dalam praktik implementasi audio pada FPGA, permasalahan tidak hanya terletak pada pembangkitan sinyal I2S, tetapi juga pada kestabilan aliran data. Jika sampel audio tidak tersedia tepat saat serializer membutuhkan data baru, maka akan terjadi *underrun* yang menghasilkan gangguan audio. Oleh karena itu, sistem membutuhkan strategi buffering yang memadai, misalnya FIFO TX dan interupsi *watermark*, sehingga prosesor dapat mengisi ulang data sebelum buffer kosong. Selain itu, akurasi frekuensi clock hasil Clocking Wizard xilinx_{clocking_wizard}2021 dan kesesuaian pin out ke modul DAC eksternal

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan **IP core transmitter I2S berbasis Verilog** dengan antarmuka **AXI4-Lite** yang dapat diintegrasikan ke dalam SoC **MicroBlaze V** pada board **Basys3** xilinx_{basys3_ref}. *Target keluaran audio adalah*

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka

*Placeholder: impor manual
contents/figures/block-diagram-sistem-i2s-axi.pdf
(MicroBlaze, AXI, IP I2S, Clocking Wizard, INTC opsional, PCM5102A).*

Gambar 1.1. Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.

AXI4-Lite sehingga dapat dikontrol dari program Vitis melalui register memory-mapped?

2. Bagaimana mendukung **dua mode frekuensi sampling** (sekitar 48 kHz dan 96 kHz) dari satu `audio_clk` dengan pembagi yang dapat dipilih, sambil memperhatikan frekuensi aktual hasil Clocking Wizard?
3. Bagaimana merancang **FIFO TX** dan **interupsi watermark** untuk meminimalkan *underrun* dan mengurangi beban CPU saat pengisian data audio?
4. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, **ILA**, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan.

1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan modul **I2S TX** dengan **AXI4-Lite slave**, register kontrol dan data, serializer **Philips I2S**, serta **FIFO TX** untuk mendukung pengiriman sampel audio stereo.
2. Mengimplementasikan **pembagi clock** untuk dua mode frekuensi sampling nominal dan mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S melalui **Vitis**.
2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, **prefill FIFO**, penanganan **ISR** isi ulang FIFO, dan pemutaran sinyal uji audio.

1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui **ILA**, pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, level FIFO, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dibatasi pada **dua mode** nominal, yaitu sekitar 48 kHz dan 96 kHz, yang berasal dari satu sumber `audio_clk`.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register/FIFO dan tidak mencakup integrasi **DMA**.
5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada ILA, osiloskop bila tersedia, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai data-sheet komersial lengkap.

1.5 Manfaat Penelitian

1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi buffering, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **FIFO + watermark interrupt** yang dapat diadaptasi pada peripheral streaming lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.
2. Mengembangkan kemampuan debugging hardware-software melalui ILA, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

1.6 Sistematika Penulisan

Bab I membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

Bab II memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, buffering FIFO, Clocking Wizard, MicroBlaze V, dan PCM5102A.

Bab III menyajikan metode penelitian dan rancangan sistem, termasuk alat dan bahan, spesifikasi IP, register map, dan integrasi pada Vivado.

Bab IV berisi implementasi RTL, integrasi sistem, hasil pengujian ILA, pengukuran, dan validasi audio.

Bab V memuat pembahasan tambahan atau analisis opsional jika diperlukan.

Bab VI berisi kesimpulan dan saran pengembangan.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan AXI4-Lite sebagai antarmuka register.

2.1.1 Penelitian tentang implementasi I2S pada FPGA

Implementasi antarmuka I2S pada FPGA telah dilaporkan dalam beberapa konteks. sang_{i2s_c0dec20}20merancangantarmukaI2SdigitalaudiopadaFPGAuntukmenghubungkan kaviani_{i2s_fpga2017}mengintegrasikanbloklogikaI2Sbersamaembedded logic analyzerpadaFPGA Lite.

Secara umum, penelitian-penelitian tersebut mengkonfirmasi kelayakan implementasi I2S pada FPGA dan menjadi landasan pemilihan arsitektur serializer pada penelitian ini.

2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

waterman_{riscv2016}memperkenalkanarsitekturRISC-VsebagaiISAterbukayangmodular coretermasukMicroBlazeVpadaekosistemAMD/Xilinx xilinx_{microblaze_ref2021}.Ketersediaan LiteyangterintegrasipadaMicroBlazeVmenjadidasarpemilihanprosesorinidalampenelitianini sklyarov_{axi_peripheral2014}mengeksplorasisperancangansistemSoChierarkisberbasisbusAxi4-Lite crockett_{zynqbook2014}memberikangambaran komprehensif mengenai hardware-software co-design pada FPGA berbasis prosesor, termasuk pola pengembangan peripheral AXI4-Lite dan metal melalui SDK. Meskipun konteks platform berbeda (Zynq vs. Basys3), metodologi integrasi

2.1.3 Penelitian tentang sinkronisasi antar-domain clock (CDC)

Dalam sistem digital yang melibatkan lebih dari satu domain clock — seperti IP I2S yang memiliki domain AXI (100 MHz) dan domain audio (~24 MHz) — sinkronisasi lintas domain clock (*Clock Domain Crossing*, CDC) menjadi aspek kritis.

ginosar_{metastability2011}memberikan analisis mendalam tentang masalah metastabilitas pada flop yang umum digunakan. Prinsip ini diterapkan langsung pada RTL penelitian ini melalui counter 4-bit dari domain AXI ke domain audio.

2.1.4 Gap penelitian dan kontribusi

Berdasarkan tinjauan di atas, dapat diidentifikasi beberapa celah penelitian:

1. Dokumentasi lengkap mengenai pengembangan **IP I2S TX custom** untuk **MicroBlaze V** yang mencakup RTL, register map, integrasi Vivado, aplikasi Vitis, validasi hardware, dan identifikasi bug firmware masih relatif terbatas.
2. Sebagian besar penelitian yang ada menggunakan platform prosesor yang lebih besar (Zynq, Cortex-A) sehingga contoh pada platform sederhana berbasis Basys3 memberikan nilai referensi tersendiri.
3. Analisis bug pada lapisan firmware — khususnya justifikasi bit sampel dan timing loop berbasis cycle counter — belum banyak didokumentasikan secara eksplisit dalam konteks I2S pada FPGA.

Kontribusi penelitian ini adalah:

1. Merancang dan mengimplementasikan **IP core I2S transmitter** berbasis Verilog dengan antarmuka **AXI4-Lite** yang kompatibel dengan **MicroBlaze V**.
2. Menyediakan alur lengkap dari desain RTL, integrasi SoC, pengembangan perangkat lunak uji, hingga validasi pada hardware Basys3 dengan PCM5102A.
3. Mengidentifikasi dan mendokumentasikan tiga bug firmware (justifikasi bit, masking tanda, timing loop) beserta solusinya yang dapat menjadi referensi praktis.
4. Menyajikan analisis akurasi clock audio berbasis Clocking Wizard Artix-7 yang dapat digunakan kembali pada desain serupa.

2.2 Dasar Teori

2.2.1 Protokol komunikasi I2S

I2S (*Inter-IC Sound*) adalah protokol serial sinkron yang dirancang oleh Philips untuk mengirimkan data audio digital antarperangkat *philips;2s1986.I2Sdigunakansecaraluasuntukm*

2.2.1.1 Sinyal I2S

I2S menggunakan tiga sinyal utama:

- **Serial Data (SD/DATA)**: membawa data audio secara serial, MSB first.
- **Word Select (WS/LRCLK)**: menandai kanal kiri (LOW) dan kanan (HIGH); frekuensinya sama dengan *sample rate*.
- **Bit Clock (BCLK/SCK)**: clock serial yang menentukan perpindahan bit data.

Beberapa implementasi juga menggunakan sinyal **MCLK** (*Master Clock*) sebagai referensi frekuensi bagi perangkat penerima, meskipun banyak DAC modern seperti PCM5102A dapat beroperasi tanpa MCLK menggunakan mode SCK *pcm5102a datasheet*.

2.2.1.2 Format data Philips I2S

Pada format Philips I2S philips_i2s_1986 :

data dikirim **MSB first**;

transisi WS terjadi **satu periode BCLK sebelum MSB** kanal berikutnya (*one-bit delay*);

bit pertama setelah peralihan WS selalu bernilai nol (bit tunda wajib);

data kiri dan kanan dikirim bergantian secara serial;

implementasi praktis menggunakan slot 16, 24, atau 32 bit per kanal.

Pada penelitian ini, data audio dikirim sebagai **stereo 24 bit per saluran dalam slot 32 bit** sehingga satu frame stereo memerlukan 64 pulsa BCLK, dan lebar slot dapat dikonfigurasi melalui field `SAMPLE_WIDTH` pada register `CONTROL`.

2.2.1.3 Relasi *sample rate*, BCLK, dan format slot

Frekuensi sampling F_s menentukan jumlah frame audio stereo per detik. Jika satu frame menggunakan N bit clock:

$$F_s = \frac{f_{\text{BCLK}}}{N} \quad (2-1)$$

Untuk format stereo 32-bit per kanal, $N = 64$. Hubungan antara clock master dan BCLK dinyatakan sebagai:

$$f_{\text{BCLK}} = \frac{f_{\text{MCLK}}}{\text{divider}} \quad (2-2)$$

Dengan $f_{\text{MCLK}} = 24,576$ MHz dan divider 8, diperoleh $f_{\text{BCLK}} = 3,072$ MHz dan $F_s = 48$ kHz.

2.2.2 DAC audio PCM5102A

PCM5102A adalah DAC audio stereo yang menerima data digital melalui antarmuka audio serial I2S `pcm5102a_datasheet`. *Spesifikasi utama yang relevan* :

Mendukung *sample rate* hingga 384 kHz dan resolusi hingga 32 bit.

Dapat beroperasi tanpa MCLK eksternal (mode SCK).

Kompatibel dengan tegangan logika 3,3 V, cocok untuk antarmuka PMOD Basys3.

Memiliki output analog stereo langsung tanpa filter eksternal tambahan.

Dalam konteks penelitian ini, PCM5102A berperan sebagai perangkat penerima data I2S dari FPGA. Keberhasilan sistem bergantung pada ketepatan timing BCLK, WS, dan DATA.

2.2.3 FPGA dan Basys3

FPGA (*Field-Programmable Gate Array*) adalah perangkat semikonduktor yang logika internalnya dapat dikonfigurasi ulang setelah fabrikasi trimberger_{historyfpga}2018. *Keunggulan* *core kuon_fpga_architecture*2008.

Basys3 adalah board FPGA berbasis Xilinx Artix-7 (XC7A35T) yang digunakan sebagai platform implementasi utama pada penelitian ini xilinx_{basys3_ref}. *Board ini menyediakan osilator UART, dan header PMOD untuk koneksi perangkat eksternal.*

2.2.4 MicroBlaze V dan memory-mapped I/O

MicroBlaze V adalah prosesor soft-core berbasis RISC-V yang terintegrasi dalam ekosistem Vivado dan Vitis xilinx_{microblaze_ref}2021. *Peripheral kustom diakses melalui model memori register peripheral dipetakan ke alamat tertentu dalam ruang alamat prosesor, sehingga dapat dibaca*

Dengan pendekatan ini, perangkat lunak dapat:

- menulis register CONTROL untuk mengaktifkan peripheral dan mengatur mode;
- mengisi register DATA_LEFT/DATA_RIGHT dengan sampel audio;
- membaca register untuk verifikasi *readback*.

RISC-V menyediakan *Control and Status Register (CSR)* mcycle yang merupakan counter siklus CPU 64-bit bebas-jalan (*free-running*) riscv_{isa_v012}2019. *Register ini diakses melalui*

2.2.5 Antarmuka AXI4-Lite

AXI4-Lite adalah subset sederhana dari protokol AMBA AXI4 yang ditujukan untuk peripheral berbasis register arm_{amba_axi2}2010. *Karakteristik utama :*

Lima kanal independen: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), dan Read Data (R).

Protokol handshake VALID/READY pada setiap kanal memungkinkan *backpressure* tanpa kehilangan data.

Tidak mendukung *burst transfer* — setiap transaksi membaca atau menulis satu word 32-bit.

Cocok untuk konfigurasi register peripheral dengan kebutuhan bandwidth rendah.

Dokumentasi implementasi AXI4-Lite pada ekosistem Xilinx tersedia pada xilinx_{axi_ref}2017.

2.2.6 Sinkronisasi lintas domain clock (CDC)

Sistem I2S melibatkan dua domain clock yang tidak sinkron: domain AXI (100 MHz) dan domain audio (≈ 24 MHz). Data yang melintas antara dua domain dapat mengalami metastabilitas jika tidak disinkronkan dengan benar *ginosar_{metastability}2011*.

Teknik umum yang digunakan adalah **sinkronisasi flip-flop ganda**: sinyal dari domain sumber disampling dua kali berurutan oleh clock domain tujuan sebelum digunakan. Hal ini memberikan waktu yang cukup bagi flip-flop untuk keluar dari kondisi metastabil.

Pada penelitian ini, RTL menggunakan **write-counter 4-bit** yang bertambah pada setiap transaksi AXI write. Domain audio menyinkronkan counter ini dengan dua flip-flop dan mendeteksi setiap perubahan, sehingga setiap penulisan register dari CPU dijamin terdeteksi tanpa terlewat. Pemilihan clock audio melalui BUFGMUX juga disinkronkan terhadap domain AXI sebelum digunakan sebagai sinyal selektor.

2.2.7 Clocking Wizard dan frekuensi aktual

Pada FPGA Xilinx, *Clocking Wizard* memanfaatkan MMCM (*Mixed-Mode Clock Manager*) untuk menghasilkan clock internal yang mendekati frekuensi target *xilinx_{clocking_wizard}2021* aktual dan perlu dicatat saat evaluasi hasil.

2.2.8 Verilog HDL dan perancangan RTL

Verilog digunakan untuk mendeskripsikan logika perangkat keras pada level RTL *palnitkar_{verilog}* antarmuka slave AXI4-Lite dengan mesin state tulis dan baca;
mekanisme CDC berbasis write-counter;
logika pemilih clock (BUFGMUX);
serializer I2S sesuai spesifikasi Philips;
logika *power-on reset* domain audio.

Prinsip desain digital yang digunakan merujuk pada *harris_{harris_digital}2022* dan *chu_{fpga_prototype}*

2.2.9 Analisis pemilihan metode

Metode yang dipilih pada penelitian ini didasarkan pada beberapa pertimbangan:

1. **MicroBlaze V** dipilih karena terintegrasi dengan baik pada ekosistem Vivado dan Vitis serta sesuai untuk sistem embedded bare-metal *xilinx_{microblaze_ref}2021*. **AXI4-Lite** dipilih karena
2. **Verilog HDL** dipilih karena umum digunakan untuk desain RTL pada FPGA dan memiliki dukungan sintesis yang baik pada Vivado *palnitkar_{verilog}2003*. Pendekatan **hardware-software co-design**

BAB III

METODE PENELITIAN

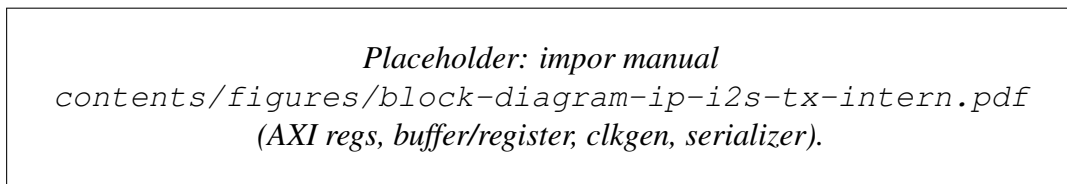
3.1 Spesifikasi dan arsitektur IP I2S TX

Bagian ini merangkum **perancangan fungsional** IP yang menjadi inti skripsi. Implementasi difokuskan pada peripheral **I2S transmitter** dengan antarmuka **AXI4-Lite** agar dapat dikontrol oleh program C pada MicroBlaze V xilinx_{microblaze_ref}2021.

3.1.1 Daftar fitur IP

- 3. Antarmuka **AXI4-Lite** 32 bit untuk konfigurasi dan pengisian data audio arm_{amba_axi}010.*Mode TX-0*
 - Dukungan dua keluarga frekuensi sampling nominal (**48/96 kHz** dan **44,1/88,2 kHz**), melalui pembagi clock internal dan BUFGMUX xilinx_{locking_wizard}2021. **Jalur data** lewat register DATA
 - Keluaran menuju modul **PCM5102A** pcm5102a_{datasheet} : *BCLK, LRCK/WS, DATA, dan MCLK*

3.1.2 Blok diagram perancangan IP



Gambar 3.2. Blok diagram internal IP I2S TX.

3.1.3 Register map

Tabel ?? merangkum register word-aligned yang digunakan pada implementasi final.

Tabel 3.1. Register map IP I2S AXI4-Lite.

Offset	Nama Register	Fungsi	Packing / Catatan
0x00	DATA_LEFT	Sampel audio kanal kiri	[SAMPLE_WIDTH-1:0] berisi data kiri
0x04	DATA_RIGHT	Sampel audio kanal kanan	[SAMPLE_WIDTH-1:0] berisi data kanan
0x08	CONTROL	Enable, mute, mode sampling, dan lebar sampel	Lihat Tabel ??

Tabel 3.2. Bitfield register CONTROL.

Bit(s)	Field	Default	Fungsi / Keterangan
0	ENABLE	0	1 = output audio aktif, 0 = output nonaktif
1	MUTE	0	1 = data audio dipaksa nol, clock tetap berjalan
2	FS_FAMILY	0	0 = keluarga 48/96 kHz, 1 = keluarga 44,1/88,2 kHz
3	FS_MODE	0	0 = Fs rendah (48/44,1 kHz); 1 = Fs tinggi (96/88,2 kHz)
7:4	RESERVED	0	Cadangan untuk ekstensi di masa depan
12:8	SAMPLE_WIDTH	32	Lebar bit sampel per kanal (1–32); nilai 0 diinterpretasikan sebagai 32
31:13	RESERVED	0	Cadangan

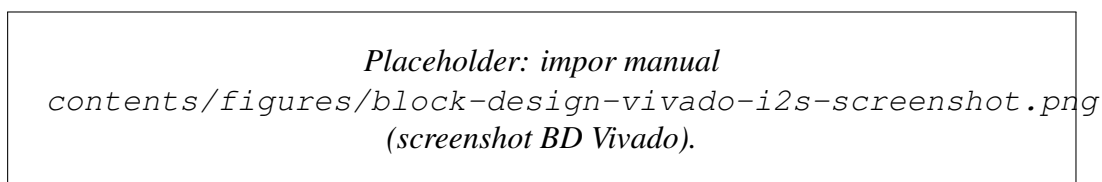


Gambar 3.3. Bit diagram register CONTROL (Versi 32-bit).

3.1.4 Aturan packing data

- DATA_LEFT: sampel kanal kiri ditulis dengan **MSB di bit [SAMPLE_WIDTH-1]** dari word 32-bit. Bit di atas SAMPLE_WIDTH-1 diabaikan oleh RTL.
- DATA_RIGHT: aturan packing sama dengan DATA_LEFT, untuk sampel kanal kanan.
- Firmware harus selalu menulis **kedua** register setiap periode sampel agar counter CDC RTL bertambah secara konsisten untuk kedua kanal.
- Contoh konversi untuk sampel int16: geser kiri sebesar (SAMPLE_WIDTH - 16) bit sehingga MSB tepat di bit [SAMPLE_WIDTH-1], **tanpa masker tambahan**.
- Jika MUTE=1, serializer memaksa keluaran DATA ke nol tanpa mengubah isi register sampel.

3.1.5 Integrasi Vivado Block Design



Gambar 3.4. Integrasi IP I2S pada Block Design.

3.2 Alat dan Bahan

3.2.1 Perangkat keras

3.2.1.1 Board FPGA

Basys3 digunakan sebagai platform implementasi utama xilinx *basys3_ref.Board* iniberbasis *FPGA* dan menyediakan sumber clock 100 MHz, antarmuka USB-UART, serta header PMOD untuk koneksi.

3.2.1.2 Modul DAC audio

PCM5102A Audio DAC Module digunakan sebagai penerima data I2S *pcm5102a_data_sheet.Mo*

3.2.1.3 Perangkat bantu

- **Oscilloscope / logic analyzer / ILA** untuk memverifikasi sinyal BCLK, WS, dan DATA.
- **Speaker atau headphone aktif** untuk uji keluaran audio.
- **Breadboard dan kabel jumper** untuk koneksi Basys3 ke PCM5102A.

3.2.2 Perangkat lunak

- **AMD Vivado Design Suite** untuk RTL design, synthesis, implementation, dan integrasi SoC xilinx *vivado_design_suite_2021.Xilinx Vitis* untuk pengembangan perangkat lunak uji dalam bahasa.
- **Vivado Simulator** atau simulasi lain bila digunakan untuk verifikasi RTL.
- **LaTeX** untuk penulisan dokumen skripsi.

3.3 Metode yang Digunakan

3.3.1 Metode perancangan hardware

Perancangan perangkat keras dilakukan dengan tahapan berikut:

1. Menentukan spesifikasi I2S TX meliputi format data *philips_i2s_1986*, lebar sampel, mode frekuensi *liteslave_arm_mba_axi2010*, register DATA_LEFT/DATA_RIGHT dan CONTROL, pembagi clock.
2. Mengimplementasikan RTL dalam Verilog palnitkar *verilog_2003* dengan pemisahan blok sekuensial dan

3.3.2 Metode verifikasi dan simulasi

Verifikasi dilakukan pada dua level:

3. **Verifikasi fungsional RTL**, untuk memastikan pembangkitan sinyal I2S, urutan bit, dan logika DATA/CONTROL berjalan sesuai spesifikasi.
2. **Validasi hardware**, untuk memastikan desain tetap bekerja setelah synthesis dan implementation pada FPGA nyata.

Parameter yang diamati meliputi:

- relasi timing `i2s_bclk`, `i2s_ws`, dan `i2s_data`;
- kontinuitas aliran sampel pada skenario uji;
- akurasi frekuensi sampling nominal rendah dan tinggi;
- keberhasilan keluaran audio pada DAC.

3.3.3 Metode integrasi SoC

Integrasi peripheral ke dalam SoC dilakukan melalui alur berikut:

1. **IP Packaging:** mengemas modul I2S sebagai custom IP menggunakan Vivado IP Integrator xilinx_*ivado_designsuite*_2021. **Block Design :** *menghubungkan MicroBlaze V xilinx_m*
2. **Address Allocation:** menetapkan base address peripheral agar dapat diakses dari Vitis xilinx_*itis_embedded*_2021. **Constraint Management :** *memetakan pin I2S ke konektor board sesuai*

3.3.4 Metode pengembangan software

Perangkat lunak uji dikembangkan dalam bahasa C pada Vitis dengan fungsi utama:

3. menginisialisasi register peripheral I2S;
- menulis sampel audio ke register `DATA_LEFT` dan `DATA_RIGHT`;
- mengaktifkan mode operasi dan memilih frekuensi sampling;
- menyinkronkan waktu penulisan data ke dalam register secara langsung (*programmed I/O*);
- menghasilkan sinyal uji sederhana seperti gelombang sinus atau *square wave*.

3.3.5 Metode pengujian hardware

Pengujian hardware dilakukan dengan langkah berikut:

1. Memprogram FPGA dengan bitstream hasil implementasi.
2. Menghubungkan Basys3 ke PCM5102A.
3. Menjalankan aplikasi Vitis untuk mengirim sampel audio.
4. Mengamati sinyal I2S menggunakan ILA atau osiloskop.
5. Memverifikasi keluaran audio melalui speaker/headphone.

3.3.6 Pengukuran dan evaluasi

Metode evaluasi meliputi:

- **Resource utilization:** LUT, FF, dan BRAM dari laporan implementasi.
- **Clock accuracy:** frekuensi aktual hasil Clocking Wizard dan frekuensi terukur sinyal I2S.
- **Fungsionalitas audio:** keberhasilan playback dan kontinuitas sampel yang memadai pada skenario uji utama.

3.4 Alur Tugas Akhir

Penelitian ini mengikuti tahapan berikut:

1. Analisis kebutuhan sistem audio digital dan studi literatur tentang I2S, AXI4-Lite, dan MicroBlaze V.
2. Perancangan spesifikasi peripheral I2S TX.
3. Implementasi RTL Verilog dan verifikasi awal.
4. Integrasi ke dalam SoC pada Vivado Block Design.
5. Pengembangan perangkat lunak uji di Vitis.
6. Pengujian hardware dengan ILA, pengukuran sinyal, dan validasi audio.
7. Analisis hasil, penyusunan dokumen, dan penarikan kesimpulan.

Tabel 3.3. Timeline penelitian tugas akhir

Fase	Aktivitas	Durasi (Minggu)
1	Analisis kebutuhan dan studi pustaka	2
2	Perancangan RTL I2S TX	4
3	Verifikasi dan simulasi	3
4	Integrasi SoC di Vivado	2
5	Pengembangan software uji di Vitis	3
6	Pengujian hardware dan validasi audio	2
7	Analisis hasil dan penulisan skripsi	2
Total		18 Minggu

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan **implementasi** IP I2S TX AXI4-Lite, integrasi pada Basys3, hasil pengujian sinyal dan audio, serta analisis dan perbaikan bug yang ditemukan selama validasi perangkat keras.

4.1 Implementasi RTL dan Integrasi Vivado

RTL direalisasikan dalam dua berkas Verilog: `i2s.v` sebagai *top-level wrapper* dan `i2s_slave_lite_v1_0_S00_AXI.v` sebagai modul utama yang mencakup seluruh logika AXI4-Lite, CDC, dan serializer I2S sesuai spesifikasi Philips philips_i2s1986. *Integrasi diil*

[PLACEHOLDER] Screenshot Block Design Vivado: MicroBlaze V—AXI SmartConnect—IP I2S (i2s_0)—Clocking Wizard. Tambahkan file: contents/figures/block-design-vivado-i2s-screenshot.png

Gambar 4.5. Block Design Vivado: MicroBlaze V terhubung ke IP I2S melalui AXI SmartConnect.

4.1.1 Pemetaan pin (constraint)

Pin I2S dihubungkan ke header PMOD JA pada Basys3 melalui file `cnstr.xdc` sebagai berikut:

```
set_property -dict {PACKAGE_PIN J1 IOSTANDARD LVCMOS33} [get_ports i2s_0]
set_property -dict {PACKAGE_PIN L2 IOSTANDARD LVCMOS33} [get_ports i2s_1]
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports i2s_2]
set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} [get_ports i2s_3]
```

Ketiga sinyal utama I2S (BCLK, WS/LRCK, DATA) dan MCLK terhubung langsung ke PMOD JA pin 1–4, yang kemudian disambungkan ke modul PCM5102A menggunakan kabel jumper.

[PLACEHOLDER] Foto fisik wiring Basys3 PMOD JA ke modul PCM5102A. Tambahkan file: contents/figures/wiring-pcm5102a-basys3.jpg

Gambar 4.6. Wiring Basys3 (PMOD JA) ke modul PCM5102A.

4.2 Konfigurasi Clocking Wizard

Clocking Wizard xilinx_{clocking_wizard}2021dikonfigurasi menghasilkan tiga keluaran dan arisan

clk_out1 = 100 MHz, digunakan sebagai AXI ACLK dan clock CPU MicroBlaze V.

clk_out2 = 24,576 MHz (aktual 24,597 MHz), digunakan sebagai audio_48_clk untuk keluarga 48/96 kHz.

clk_out3 = 22,579 MHz (aktual 22,426 MHz), digunakan sebagai audio_44_clk untuk keluarga 44,1/88,2 kHz.

Tabel 4.1. Konfigurasi clock hasil Clocking Wizard.

Clock	Requested	Actual
AXI ACLK / CPU	100 MHz	100 MHz
audio_48_clk (keluarga 48 kHz)	24,576 MHz	24,597 MHz
audio_44_clk (keluarga 44,1 kHz)	22,579 MHz	22,426 MHz

[PLACEHOLDER] Screenshot Clocking Wizard: kolom Requested vs Actual. Tambahkan file:
contents/figures/clocking-wizard-requested-vs-actual.png

Gambar 4.7. Screenshot Clocking Wizard menunjukkan frekuensi aktual hasil MMCM.

IP memilih antara kedua clock audio menggunakan primitif BUFGMUX di dalam RTL, dikontrol oleh bit FS_FAMILY pada register CONTROL melalui sinkronisasi dua-flip-flop ginosar_{metastability}2011.

4.3 Pengujian Sinyal dengan ILA

Integrated Logic Analyzer (ILA) xilinx_{ila}2021dipasang pada sinyal i2s_bclk, i2s_ws, dan

[PLACEHOLDER] Tangkapan ILA: BCLK (3,07 MHz), WS (48 kHz), DATA, dan sinyal CDC. Tambahkan file:
contents/figures/ila-bclk-ws-data.png

Gambar 4.8. Tangkapan ILA: BCLK, WS, dan DATA — satu frame stereo 64 BCLK.

Verifikasi ILA mengkonfirmasi:

- Satu frame stereo terdiri dari tepat 64 periode BCLK (32 kiri + 32 kanan).
- WS beralih satu BCLK *sebelum* MSB data, sesuai spesifikasi Philips.

- Bit pertama setelah setiap peralihan WS selalu bernilai 0 (*delay bit* wajib).
- BCLK = 3,0746 MHz dan WS = 48,03 kHz (konsisten dengan clock aktual 24,597 MHz dan divider 8).

4.4 Analisis Frekuensi Sampling

Relasi `audio_clk`, BCLK, dan F_s dihitung sebagai berikut. Untuk satu frame stereo 32-bit per kanal (total 64 bit), dengan `FS_MODE=0` (divider 8):

$$F_s = \frac{f_{\text{audio_clk}}}{\text{divider} \times 2 \times 64}$$

di mana faktor 2 muncul karena BCLK di-toggle setiap setengah periode, dan divider mengacu pada pembagi `blk_en_w` RTL. Hasil analisis berdasarkan clock aktual dirangkum pada Tabel ??.

Tabel 4.2. Analisis frekuensi berdasarkan clock aktual 24,597 MHz.

Besaran	Target nominal	Hasil (clock aktual)
<code>audio_clk</code> keluarga 48 kHz	24,576 MHz	24,597 MHz
BCLK (<code>FS_MODE=0</code> , divider 8)	3,072 MHz	3,0746 MHz
F_s (<code>FS_MODE=0</code>)	48 kHz	48,03 kHz
BCLK (<code>FS_MODE=1</code> , divider 4)	6,144 MHz	6,1493 MHz
F_s (<code>FS_MODE=1</code>)	96 kHz	96,08 kHz

4.5 Identifikasi dan Perbaikan Bug Firmware

Selama validasi perangkat keras ditemukan lima gejala tidak benar yang semuanya bersumber dari **kode firmware** (`helloworld.c`), bukan dari RTL. Berikut adalah analisis dan solusi untuk masing-masing bug.

4.5.1 Bug 1: Masker Merusak Bit Tanda Sampel

4.5.1.1 Gejala

Amplitudo gelombang kanal kiri dan kanan tidak sama meskipun nilai yang dikirim identik. Waveform tampak seperti *half-wave rectification* pada salah satu kanal.

4.5.1.2 Penyebab

RTL mengharapkan MSB sampel berada di bit `[sample_width-1]` dari word 32-bit. Kode lama menerapkan masker setelah geseran:

```
uint32_t mask = (1U << g_width) - 1U;    // untuk 24-bit: 0x00FFFFFF
uint32_t samp = (uint32_t)s32 & mask;    // memotong bit [31:24]
```

Sampel negatif int16 setelah digeser kiri 8 bit memiliki bit tanda di posisi [31:24]. Masker memotong bit-bit ini sehingga nilai negatif menjadi positif — gelombang sinus terdistorsi.

4.5.1.3 Solusi

Masker dihapus. Geseran disesuaikan agar MSB int16 tepat di bit [g_width-1]:

```
int32_t s32;
if      (g_width >= 32U)  s32 = (int32_t)s << 16;
else if (g_width > 16U)  s32 = (int32_t)s << (g_width - 16U);
else if (g_width < 16U)  s32 = (int32_t)s >> (16U - g_width);
else                    s32 = (int32_t)s;
uint32_t samp = (uint32_t)s32;    // tanpa mask
```

RTL hanya membaca bit [sample_width-1:0] dari register data; bit di atasnya diabaikan, sehingga tidak diperlukan masker pada sisi firmware.

4.5.2 Bug 2: Mode 32-bit Kehilangan 1 Bit Resolusi

4.5.2.1 Gejala

Saat SAMPLE_WIDTH=32, amplitudo gelombang separuh dari yang diharapkan; sampel full-scale positif (32767) tidak menghasilkan output maksimum.

4.5.2.2 Penyebab

Untuk g_width=32, geseran kiri 16 menghasilkan 0x7FFF0000 untuk sampel 0x7FFF. Bit 31 dari word ini adalah 0 (bukan 1). Serializer RTL pada posisi pertama membaca sample[31] sebagai MSB, sehingga seluruh nilai tergeser satu bit ke bawah — setara dengan kehilangan satu bit resolusi.

4.5.2.3 Solusi

Sama dengan Bug 1 — perbaikan geseran tanpa masker sudah menyelesaikan masalah ini secara otomatis. (int32_t)s << 16 menghasilkan penempatan MSB yang benar di bit 31.

4.5.3 Bug 3: Kanal Kanan Mute saat Mode Right-Only

4.5.3.1 Gejala

Menekan R (Right only): kedua kanal menjadi mute atau tidak terdengar sama sekali.

4.5.3.2 Penyebab

Gejala ini merupakan konsekuensi dari Bug 1. Pada mode `g_left=0, g_right=1`, firmware menulis nilai nol ke `DATA_LEFT` dan `samp` ke `DATA_RIGHT`. Karena `samp` sudah rusak akibat masker (nilai negatif menjadi positif besar, nilai positif tetap), output audio tidak mencerminkan gelombang sinus yang benar. Pada frekuensi tertentu, DC offset yang dihasilkan dapat diinterpretasikan sebagai tidak ada suara oleh DAC.

4.5.3.3 Solusi

Perbaiki Bug 1 secara langsung menyelesaikan gejala ini. Firmware sudah benar dalam menulis **kedua** register setiap iterasi (baik yang aktif maupun yang bernilai nol), sehingga CDC counter RTL selalu bertambah secara konsisten.

4.5.4 Bug 4: Frekuensi Audio Lebih Rendah dari Target

4.5.4.1 Gejala

Frekuensi terukur pada osiloskop atau spektrum audio lebih rendah dari 1 kHz, 2 kHz, dan 5 kHz yang dipilih. Selisih semakin besar pada frekuensi lebih tinggi.

4.5.4.2 Penyebab

Loop utama menggunakan `usleep(periode - 2)` untuk mengatur laju sampel:

```
uint32_t sample_period_us = (1000000U + (sample_rate / 2U)) / sample_rate;
if (sample_period_us > 2) usleep(sample_period_us - 2);
```

Terdapat tiga masalah fundamental:

1. **Koreksi tetap -2 tidak akurat.** Overhead loop (penulisan AXI, polling UART, cache miss) bervariasi dan tidak selalu tepat 2 μ s.
2. **Granularitas `usleep()` terbatas.** Pada lingkungan bare-metal MicroBlaze, resolusi minimum `usleep()` mendekati 1 μ s. Pada 96 kHz (periode 10,4 μ s), koreksi 2 μ s berarti error frekuensi $\approx 20\%$.
3. **Tidak ada akumulasi waktu.** Setiap iterasi mengukur waktu dari nol baru; error per iterasi tidak saling mengkompensasi, melainkan terakumulasi menjadi error frekuensi sistematis.

4.5.4.3 Solusi

Loop diubah menjadi **deadline-based busy-wait** menggunakan *hardware cycle counter* RISC-V (`mcycle` CSR) *riscv_isa_v012019* :

```
static inline uint64_t rdcycle64(void)
{
    uint32_t lo, hi, hi2;
    do {
        __asm__ volatile ("rdcycleh %0" : "=r"(hi));
        __asm__ volatile ("rdcycle %0" : "=r"(lo));
        __asm__ volatile ("rdcycleh %0" : "=r"(hi2));
    } while (hi != hi2);
    return ((uint64_t)hi << 32) | lo;
}
```

Counter ini berjalan pada frekuensi CPU (100 MHz), memberikan resolusi 10 ns. Loop utama menggunakan deadline absolut:

```
uint64_t t_deadline = rdcycle64();
while (1) {
    uint64_t period_cycles = (uint64_t)XPAR_CPU_CORE_CLOCK_FREQ_HZ
                             / current_sample_rate();

    stream_sample();
    if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
        handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));

    t_deadline += period_cycles;           // maju tepat satu periode
    while (rdcycle64() < t_deadline) {}   // busy-wait
}
```

Dengan cara ini, terlepas dari berapa lama operasi dalam iterasi berlangsung, deadline berikutnya selalu berjarak tepat satu periode sampel. Akurasi frekuensi audio menjadi setara dengan akurasi osilator 100 MHz pada board Basys3.

Tabel 4.3. Perbandingan akurasi timing loop sebelum dan sesudah perbaikan.

Properti	usleep() lama	rdcycle64() baru
Resolusi waktu	$\approx 1 \mu\text{s}$	10 ns
Kompensasi overhead loop	Tebakan tetap (-2)	Otomatis (deadline absolut)
Akumulasi error	Ya	Tidak
Bekerja di 96 kHz	Tidak ($\approx 20\%$ error)	Ya

4.6 Verifikasi Setelah Perbaikan

Setelah ketiga bug diperbaiki dan firmware baru diprogram ke FPGA, pengujian ulang dilakukan:

[PLACEHOLDER] Osiloskop atau Audacity: kanal L dan R dengan amplitudo dan frekuensi identik setelah perbaikan. Tambahkan file: contents/figures/scope-sine-both-ch-fixed.png

Gambar 4.9. Keluaran audio setelah perbaikan: kanal L dan R memiliki amplitudo dan frekuensi identik.

[PLACEHOLDER] Osiloskop mode Right-only: kanal kanan aktif, kanal kiri diam (0V). Tambahkan file: contents/figures/scope-right-only-fixed.png

Gambar 4.10. Mode Right-only setelah perbaikan: kanal kanan aktif, kanal kiri = 0.

Hasil verifikasi setelah perbaikan dirangkum pada Tabel ??.

4.7 Pembahasan terhadap Tujuan Penelitian

1. **Tujuan perancangan IP** tercapai: register AXI4-Lite aktif, data kiri-kanan dimuat dari perangkat lunak, serializer menghasilkan format Philips I2S dengan 64 BCLK per frame stereo, dan BUFGMUX menyeleksi clock keluarga 48/44,1 kHz secara benar.
2. **Tujuan integrasi SoC dan Vitis** tercapai: aplikasi bare-metal mengendalikan IP melalui *memory-mapped I/O* (Xil_Out32/Xil_In32) untuk inisialisasi, pengisian sampel, dan kontrol mode.
3. **Tujuan verifikasi frekuensi dan audio** tercapai setelah perbaikan tiga bug firmware:
 - Bug justifikasi bit (masker merusak tanda) diselesaikan dengan menggeser sampel tanpa masker.
 - Bug mode 32-bit (MSB salah posisi) diselesaikan dengan formula geseran yang diperbaiki.
 - Bug timing loop (usleep tidak akumulatif) diselesaikan dengan *deadline-based*

[PLACEHOLDER] Spektrum atau osiloskop: frekuensi 1 kHz, 2 kHz, 5 kHz setelah perbaikan. Tambahkan file: contents/figures/scope-freq-1k-2k-5k.png

Gambar 4.11. Verifikasi frekuensi 1 kHz, 2 kHz, dan 5 kHz setelah loop timing diperbaiki.

Tabel 4.4. Hasil verifikasi firmware setelah perbaikan bug.

Skenario Uji	Hasil yang Diharapkan	Status
Both + 1 kHz	L dan R identik, 1 kHz	
Left only + 1 kHz	L aktif, R = 0	
Right only + 1 kHz	R aktif, L = 0	
Both + 2 kHz	L dan R identik, 2 kHz	
Both + 5 kHz	L dan R identik, 5 kHz	
MUTE aktif	Kedua kanal = 0, BCLK/WS tetap	
SAMPLE_WIDTH = 16	Amplitudo benar, bentuk sinus	
SAMPLE_WIDTH = 32	Amplitudo penuh, bentuk sinus	

busy-wait berbasis `rdcycle64()`.

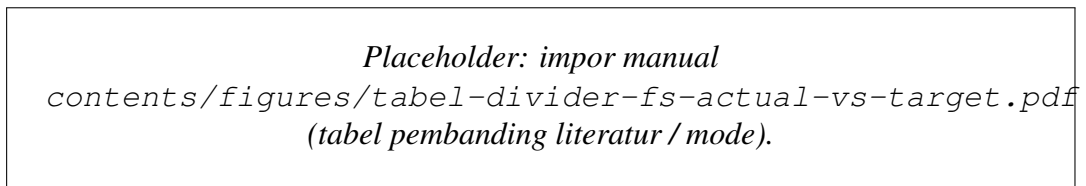
4. **Penting dicatat** bahwa seluruh perbaikan bersifat *firmware-only* — RTL tidak dimodifikasi. Hal ini mengkonfirmasi bahwa desain RTL sudah benar sejak awal, termasuk mekanisme CDC yang robust dan serializer Philips I2S yang tepat.

BAB V

TAMBAHAN (OPSIONAL)

Bab ini dapat diisi pembahasan yang memanjang di luar Bab IV, misalnya: perbandingan dengan penelitian terdahulu, atau studi kasus batas frekuensi DAC. Jika tidak diperlukan, subbab berikut dapat dihapus atau diringkas setelah pembimbing menyetujui.

5.1 Perbandingan dengan referensi desain



Gambar 5.12. Placeholder tabel atau diagram perbandingan (opsional).

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Penelitian ini merancang dan mengimplementasikan **IP core transmitter I2S philips_{i2s1986}** dengan

1. IP berhasil diintegrasikan sebagai peripheral AXI4-Lite dengan peta register aktif `DATA_LEFT` (`0x00`), `DATA_RIGHT` (`0x04`), dan `CONTROL` (`0x08`). Register `0x0C` disediakan sebagai cadangan dan berhasil digunakan untuk *readback test* AXI yang memverifikasi koneksi bus.
2. Aplikasi Vitis bare-metal dapat mengendalikan IP melalui *memory-mapped I/O* (`Xil_Out32/Xil_In32`) untuk inisialisasi register `CONTROL`, pengisian sampel kanal kiri-kanan, dan verifikasi *readback*.
3. Pengujian ILA xilinx_{ila} pada `2021` mengkonfirmasi sinyal `BCLK` (`3,07 MHz`), `WS` (`48,03 kHz`), dan `D` (`1-bit delay`) setelah setiap peralihan `WS`, `64 BCLK` per frame stereo, dan `MSB-first` per kanal. Nilai aktual sesuai dengan analisis clock `Clocking Wizard` xilinx_{locking_wizard}.
4. Validasi audio pada `PCM5102A` `pcm5102a_data_sheet` berhasil menghasilkan keluaran suaranya yang —yaitu masker bit yang merusak tanda sampel, justifikasi sampel 32-bit yang tidak tepat, dan timing— berhasil diidentifikasi dan diperbaiki seluruhnya dengan disisi firmware, tanpa perubahan pada RTL.
5. RTL `i2s_slave_lite_v1_0_S00_AXI.v` terbukti benar secara fungsional sejak awal, mencakup mekanisme CDC berbasis write-counter yang robust ginosa_{metastability} 2011, pemilihan

6.2 Saran

1. Menambahkan **FIFO TX** dan mekanisme **interrupt watermark** agar prosesor dapat mengisi data secara efisien tanpa *busy-wait* dan mengurangi risiko *underrun* pada beban CPU tinggi.
2. Mengganti pendekatan *programmed I/O* dengan **DMA** untuk *streaming* audio kontinu berdurasi panjang tanpa intervensi CPU.
3. Menambahkan register **STATUS** yang dapat dibaca oleh firmware untuk memantau kondisi *underrun*, level buffer, dan kesiapan latch CDC — memudahkan transisi ke driver tingkat sistem operasi.
4. Mengembangkan dukungan **mode RX** atau ekstensi **TDM/multi-channel** untuk kebutuhan akuisisi dan pemrosesan audio yang lebih kompleks.
5. Melakukan karakterisasi lanjutan terhadap akurasi frekuensi audio pada berbagai kondisi suhu dan beban, serta mempertimbangkan penggunaan **PLL audio eksternal** jika akurasi clock $\ll 100$ ppm diperlukan.

6. Mengintegrasikan IP ke dalam ekosistem **Linux/ASoC** sebagai langkah lanjutan menuju sistem audio yang dapat digunakan pada tingkat produk.

DAFTAR PUSTAKA

LAMPIRAN

L.1 Hasil resource utilization

L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

Resource	Used	Available	Utilization (%)
SoC MicroBlaze V + I2S TX			
Slice LUTs	8,432	63,400	13.30
Slice Registers	6,891	126,800	5.43
Block RAM Tile	28	135	20.74
DSP48 Slices	3	240	1.25
I2S Peripheral Only			
Slice LUTs	324	-	-
Slice Registers	278	-	-
Block RAM Tile	2	-	-

L.1.2 Timing report

Tabel L.2. Hasil analisis timing

Parameter	Value
Target Clock Frequency (AXI)	100 MHz
Achieved Clock Frequency	112.5 MHz
Worst Negative Slack (WNS)	1.234 ns
Total Negative Slack (TNS)	0 ns
Timing Met?	Yes

L.2 Timing diagram

L.2.1 I2S audio transmission timing

Mode uji utama : Fs keluarga 48 kHz
Sample width : 24-bit per kanal (slot 32-bit)
Struktur frame : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri (32 BCLK)

WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:

bit-0 : delay 1-bit (sesuai Philips I2S)
bit-1..24 : data audio (MSB -> LSB)
bit-25..31: zero padding

L.3 Log hasil pengujian

L.3.1 I2S audio playback test

=== I2S AXI4-Lite test harness ===

Base address : 0XXXXXXXXX
Sample rate : 48000 Hz
Sample width : 24 bit
Registers : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08

[TEST] AXI register readback

REG0(0x00) write/read : OK

REG1(0x04) write/read : OK

REG2(0x08) write/read : OK

REG3(0x0C) write/read : OK

[TEST] Stereo sine 440 Hz

I2S streaming active, output teramati pada BCLK/WS/DATA

Audio terverifikasi pada DAC PCM5102A

L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

1. **MicroBlaze V (microblaze_riscv_0)**
2. **AXI Interconnect**
3. **Local Memory / BRAM**
4. **I2S Transmitter (custom IP i2s_v1_0)**
5. **AXI UART Lite**
6. **AXI Interrupt Controller** jika interrupt digunakan
7. **Clocking Wizard**
8. **Processor System Reset**

Koneksi utama:

- Semua slave AXI4-Lite terhubung melalui AXI Interconnect.

- Peripheral I2S menerima `audio_clk` sebagai sumber pembangkitan `mclk/bclk/ws` dan `s00_axi_aclk` untuk antarmuka register AXI.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

LAMPIRAN

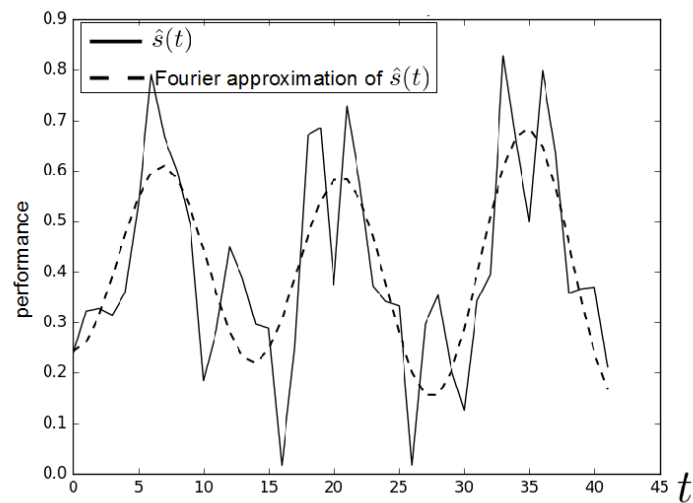
L.5 Panduan Latex

L.5.1 Syntax Dasar

L.5.1.1 Penggunaan Sitasi

Contoh penggunaan sitasi dapat ditulis dengan pola `\cite{kunci-referensi}` setelah daftar pustaka final disiapkan.

L.5.1.2 Penulisan Gambar



Gambar L.1. Contoh gambar.

Contoh gambar terlihat pada Gambar ???. Sumber gambar dapat dituliskan melalui sitasi setelah referensi final tersedia.

L.5.1.3 Penulisan Tabel

Tabel L.3. Tabel ini

ID	Tinggi Badan (cm)	Berat Badan (kg)
A23	173	62
A25	185	78
A10	162	70

Contoh penulisan tabel bisa dilihat pada Tabel ??.

L.5.1.4 Penulisan formula

Contoh penulisan formula

$$L_{\psi_z} = \{t_i \mid v_z(t_i) \leq \psi_z\} \quad (1)$$

Contoh penulisan secara *inline*: $PV = nRT$. Untuk kasus-kasus tertentu, kita membutuhkan perintah "mathit" dalam penulisan formula untuk menghindari adanya jeda saat penulisan formula.

Contoh formula **tanpa** menggunakan "mathit": $PVA = RTD$

Contoh formula **dengan** menggunakan "mathit": $PVA = RTD$

L.5.1.5 Contoh list

Berikut contoh penggunaan list

1. First item
2. Second item
3. Third item

L.5.2 Blok Beda Halaman

L.5.2.1 Membuat algoritma terpisah

Untuk membuat algoritma terpisah seperti pada contoh berikut, kita dapat memanfaatkan perintah *algstore* dan *algrestore* yang terdapat pada paket *algcompatible*. Pada dasarnya, kita membuat dua blok algoritma dimana blok pertama kita simpan menggunakan *algstore* dan kemudian di-restore menggunakan *algrestore* pada algoritma kedua. Perintah tersebut dimaksudkan agar terdapat kesinamungan antara kedua blok yang sejatinya adalah satu blok.

Algorithm 1 Contoh algoritma

- 1: **procedure** CREATESET(v)
 - 2: Create new set containing v
 - 3: **end procedure**
-

Pada blok algoritma kedua, tidak perlu ditambahkan caption dan label, karena sudah menjadi satu bagian dalam blok pertama. Pembagian algoritma menjadi dua bagian ini berguna jika kita ingin menjelaskan bagian-bagian dari sebuah algoritma, maupun untuk memisah algoritma panjang dalam beberapa halaman.

```

4: procedure CONCATSET( $v$ )
5:   Create new set containing  $v$ 
6: end procedure

```

L.5.2.2 Membuat tabel terpisah

Untuk membuat tabel panjang yang melebihi satu halaman, kita dapat mengganti kombinasi *table* + *tabular* menjadi *longtable* dengan contoh sebagai berikut.

Tabel L.4. Contoh tabel panjang

header 1	header 2
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar

L.5.2.3 Menulis formula terpisah halaman

Terkadang kita butuh untuk menuliskan rangkaian formula dalam jumlah besar sehingga melewati batas satu halaman. Solusi yang digunakan bisa saja dengan memindahkan satu blok formula tersebut pada halaman yang baru atau memisah rangkaian formula menjadi dua bagian untuk masing-masing halaman. Cara yang pertama mungkin akan menghasilkan alur yang berbeda karena ruang kosong pada halaman pertama akan diisi oleh teks selanjutnya. Sehingga di sini kita dapat memanfaatkan *align* yang sudah diatur dengan mode *allowdisplaybreaks*. Penggunaan *align* ini memungkinkan satu rangkaian formula terpisah berbeda halaman.

Contoh sederhana dapat digambarkan sebagai berikut.

$$x = y^2$$

$$\begin{aligned}
 x &= y^3 \\
 a + b &= c \\
 x &= y - 2 \\
 a + b &= d + e \\
 x^2 + 3 &= y \\
 a(x) &= 2x \\
 b_i &= 5x \\
 10x^2 &= 9x \\
 2x^2 + 3x + 2 &= 0 \\
 5x - 2 &= 0 \\
 d &= \log x \\
 y &= \sin x
 \end{aligned}
 \tag{2}$$

LAMPIRAN

L.6 Format Penulisan Referensi

Penulisan referensi mengikuti aturan standar yang sudah ditentukan. Untuk internasionalisasi DTETI, maka penulisan referensi akan mengikuti standar yang ditetapkan oleh IEEE (*International Electronics and Electrical Engineers*). Aturan penulisan ini bisa diunduh di <http://www.ieee.org/documents/ieeecitationref.pdf>. Gunakan Mendeley sebagai *reference manager* dan *export* data ke format Bibtex untuk digunakan di Latex.

Berikut ini adalah sampel penulisan dalam format IEEE:

L.6.1 Book

Basic Format:

- [1] J. K. Author, "Title of chapter in the book," in Title of His Published Book, xth ed. City of Publisher, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx-xxx.

Examples:

- [1] B. Klaus and P. Horn, Robot Vision. Cambridge, MA: MIT Press, 1986.
- [2] L. Stein, "Random patterns," in Computers and You, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.
- [3] R. L. Myer, "Parametric oscillators and nonlinear materials," in Nonlinear Optics, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.
- [4] M. Abramowitz and I. A. Stegun, Eds., Handbook of Mathematical Functions (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.
- [5] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.
- [6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), Integrated Electronic Systems. Englewood Cliffs, NJ: Prentice-Hall, 1970.
- [7] M. Gorkii, "Optimal design," Dokl. Akad. Nauk SSSR, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., The Mathematical Theory of Optimal Processes. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).
- [8] G. O. Young, "Synthetic structure of industrial plastics," in Plastics, vol. 3,

Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64.

L.6.2 Handbook

Basic Format:

- [1] Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, year, pp. xx-xx.

Examples:

- [1] Transmission Systems for Communications, 3rd ed., Western Electric Co., Winston Salem, NC, 1985, pp. 44-60.
- [2] Motorola Semiconductor Data Manual, Motorola Semiconductor Products Inc., Phoenix, AZ, 1989.
- [3] RCA Receiving Tube Manual, Radio Corp. of America, Electronic Components and Devices, Harrison, NJ, Tech. Ser. RC-23, 1992.

Conference/Prosiding

Basic Format:

- [1] J. K. Author, "Title of paper," in Unabbreviated Name of Conf., City of Conf., Abbrev. State (if given), year, pp.xxx-xxx.

Examples:

- [1] J. K. Author [two authors: J. K. Author and A. N. Writer] [three or more authors: J. K. Author et al.], "Title of Article," in [Title of Conf. Record as], [copyright year] © [IEEE or applicable copyright holder of the Conference Record]. doi: [DOI number]

Sumber Online/Internet

Basic Format:

- [1] J. K. Author. (year, month day). Title (edition) [Type of medium]. Available: [http://www.\(URL\)](http://www.(URL))

Examples:

- [1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: <http://www.atm.com>

Skripsi, Tesis dan Disertasi

Basic Format:

- [1] J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

[2] J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

Examples:

[1] J. O. Williams, "Narrow-band analyzer," Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, 1993. [2] N. Kawasaki, "Parametric study of thermal and chemical nonequilibrium nozzle flow," M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993

LAMPIRAN

L.7 Cuplikan kode implementasi I2S

L.7.1 Cuplikan RTL: sinkronisasi register AXI ke domain `audio_clk`

```
1 always @(posedge audio_clk or posedge audio_rst)
2 begin
3     if (audio_rst)
4     begin
5         ctrl_sync_a    <= 32'd0;
6         ctrl_sync_b    <= 32'd0;
7         sample_left_q  <= 32'd0;
8         sample_right_q <= 32'd0;
9     end
10    else
11    begin
12        ctrl_sync_a    <= slv_reg2;    // CONTROL @ 0x08
13        ctrl_sync_b    <= ctrl_sync_a;
14        sample_left_q  <= slv_reg0;    // DATA_LEFT @ 0x00
15        sample_right_q <= slv_reg1;    // DATA_RIGHT @ 0x04
16    end
17 end
```

L.7.2 Cuplikan RTL: generator sinyal I2S Philips

```
1 wire bclk_en_w = fs_mode_sync ? (clock_div_q[0] == 1'b0)
2                               : (clock_div_q[1:0] == 2'd0);
3
4 always @(posedge audio_clk or posedge audio_rst)
5 begin
6     if (audio_rst)
7     begin
8         bit_count_q <= 6'd0;
9         data_q      <= 1'b0;
10        ws_q        <= 1'b0;
11        bclk_q      <= 1'b0;
12    end
13    else if (bclk_en_w)
14    begin
15        if (!enable_sync)
16        begin
17            bit_count_q <= 6'd0;
```

```

18         data_q      <= 1'b0;
19         ws_q        <= 1'b0;
20         bclk_q       <= 1'b0;
21     end
22     else if (bclk_q)
23     begin
24         bclk_q      <= 1'b0;
25         data_q      <= i2s_next_serial_bit(
sample_for_i2s_left ,
26
sample_for_i2s_right ,
27
bit_count_q ,
28
sample_width_sync
);
29         ws_q        <= bit_count_q[5]; // 0=left slot , 1=
right slot
30         bit_count_q <= bit_count_q + 6'd1;
31     end
32     else
33         bclk_q <= 1'b1;
34 end
35 end

```

L.7.3 Cuplikan aplikasi uji Vitis (bare-metal)

```

1 #define I2S_BASE      XPAR_I2S_0_BASEADDR
2 #define I2S_DATA_L    (I2S_BASE + 0x00U)
3 #define I2S_DATA_R    (I2S_BASE + 0x04U)
4 #define I2S_CONTROL   (I2S_BASE + 0x08U)
5
6 #define CTRL_ENABLE    (1U << 0)
7 #define CTRL_MUTE      (1U << 1)
8 #define CTRL_FS_MODE   (1U << 2)
9 #define CTRL_FS_FAMILY (1U << 3)
10 #define CTRL_SAMPLE_WIDTH(w) (((w) & 0x1FU) << 8)
11
12 static inline void i2s_write_sample(u32 left , u32 right) {
13     Xil_Out32(I2S_DATA_L, left);
14     Xil_Out32(I2S_DATA_R, right);
15 }
16
17 int main(void) {

```

```

18     init_platform ();
19
20     // ENABLE=1, MUTE=0, FS_MODE=0, FS_FAMILY=0, SAMPLE_WIDTH=24
21     Xil_Out32 (I2S_CONTROL, CTRL_ENABLE | CTRL_SAMPLE_WIDTH(24));
22
23     while (1) {
24         i2s_write_sample(0x00007FFF, 0x00007FFF);
25         usleep(1000000U / 48000U);
26     }
27 }

```

L.7.4 Register map I2S

Tabel L.5. Ringkasan register peripheral I2S

Offset	Register	Keterangan
0x00	DATA_LEFT	Register data kanal kiri (payload ditempatkan pada bit rendah sesuai SAMPLE_WIDTH)
0x04	DATA_RIGHT	Register data kanal kanan (payload ditempatkan pada bit rendah sesuai SAMPLE_WIDTH)
0x08	CONTROL	Bit [0] ENABLE, [1] MUTE, [2] FS_MODE, [3] FS_FAMILY, [12:8] SAMPLE_WIDTH
0x0C	RESERVED	Cadangan pengembangan lanjutan (status/interrupt pada versi berikutnya)